

Amazones – Python

Rapport préliminaire

Messaoudi Ismail, Mohammed Amine Rabih , Mohamed Ali Chaoui, Ayoub oubakki, Mahmoud Mounouar

29 janvier 2026

Introduction

Ce projet s’inscrit dans le cadre du module Projet de Programmation du Master Informatique 2025–2026. Il vise à concevoir une application logicielle permettant de jouer au jeu de plateau *Amazones*, un jeu à deux joueurs, déterministe et à information parfaite.

Le jeu des Amazones combine des éléments des échecs et du Go. Malgré des règles simples, il possède une grande profondeur stratégique. Sa complexité PSPACE-complète [3] en fait un support pertinent pour l’étude des algorithmes d’intelligence artificielle. Plusieurs solveurs performants [7] ont été développés pour ce jeu, démontrant l’intérêt de la communauté scientifique.

Spécifications techniques

Élément	Choix
Langage	Python 3.10+
Build-system	pyproject.toml + setuptools
Interface graphique	PyGObject (GTK 4.x)
Framework de tests	pytest + coverage ($\geq 85\%$)
Documentation	Sphinx
Internationalisation	gettext (anglais par défaut, français)
Machine Learning	Keras
Style de codage	PEP8

TABLE 1 – Stack technique du projet

1 Présentation du jeu des Amazones

1.1 Règles du jeu et Spécifications Algorithmiques

Le jeu se déroule sur un plateau de taille 10×10 (configurable de 4×4 à 11×11). Chaque joueur contrôle quatre amazones (sauf pour les petits plateaux 4×4 et 5×5).

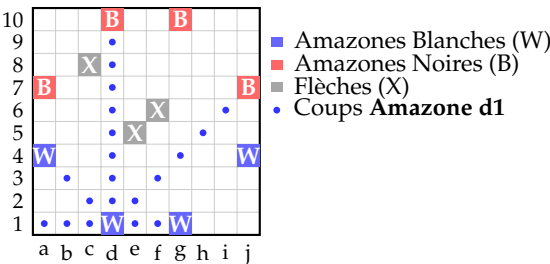


FIGURE 1 – Plateau de jeu 10×10 avec positions initiales et visualisation des cases accessibles (points) pour l’amazone en d1.

1.1.1 Algorithme de Déplacement

Pour garantir une implémentation robuste, le déplacement d'une Amazone (et le tir de flèche) suit une logique vectorielle :

1. L'algorithme parcourt les 8 directions cardinales et intercardinales :

$$DIRS = \{(1, 0), (-1, 0), (0, 1), (0, -1), (1, 1), (-1, -1), (1, -1), (-1, 1)\}$$

2. Pour chaque direction $\vec{d} \in DIRS$, on progresse case par case tant que la case suivante est **VIDE** (EMPTY).
3. **Condition d'arrêt** : la propagation s'arrête dès qu'un obstacle (Amazone ou Flèche) ou le bord du plateau est atteint.

Note technique : Cette logique constitue la définition fonctionnelle. L'implémentation technique traduira ces vecteurs en masques binaires pour le module Bitboard (voir section Structures de données) afin d'optimiser les performances.

1.1.2 Conditions de Fin de Partie (F10)

La partie se termine lorsqu'un joueur ne peut plus effectuer de coup légal (mouvement + tir). Ce joueur perd la partie.

Cas particuliers :

- **Séparation du plateau** : si le plateau est divisé en zones isolées (Amazones blanches d'un côté, Noires de l'autre), le jeu **continue**.
- **Course de remplissage** : chaque joueur joue dans son territoire jusqu'à épuisement total des coups possibles.
- **Déterminisme** : le jeu des Amazones est fini et déterministe. Le nombre de cases libres diminue strictement de 1 à chaque tour (tir de flèche). Le match nul est impossible : il existe toujours un vainqueur.

1.1.3 Gestion des Erreurs et Validité (F11)

Le moteur de jeu doit garantir qu'aucun coup illégal n'altère l'état de la partie. Conformément à l'exigence **F11** :

- Tout coup ne respectant pas l'algorithme de déplacement (traversée d'obstacle, case d'arrivée occupée) est immédiatement rejeté.
- Un message d'erreur explicite est retourné à l'interface (ex : "*Coup invalide : Trajectoire bloquée en d4*").
- Le joueur est invité à recommencer sa saisie sans pénalité.

1.2 Placement initial des amazones (grille réglable)

Problème : Comment placer les amazones sur une grille de taille variable $N \times N$ tout en garantissant la jouabilité ? Conserver quatre amazones sur de petits plateaux entraîne une saturation excessive de l'espace (densité $> 30\%$).

Nous adoptons une stratégie adaptative basée sur la réduction du nombre de pièces pour les petites grilles et une formule géométrique proportionnelle pour les tailles standards.

Taille ($N \times N$)	Nb Amazones	Stratégie de placement
4×4	2	Coins opposés (Densité 25%)
5×5	3	Triangle (Densité 24%)
6×6 à 11×11	4	Formule des Tiers (Densité 22% à 8%)

TABLE 2 – Stratégies de placement adaptatives selon la taille

1.2.1 Algorithme des Tiers (pour $N \geq 6$)

Pour placer les 4 amazones de manière équilibrée, on calcule l'indice de tiers :

$$T = \left\lfloor \frac{N}{3} \right\rfloor$$

Les positions du joueur **Blanc** (bas du plateau — Ligne 0) sont définies comme suit :

- **Ailes** : Ligne T , Colonnes 0 et $N - 1$.
- **Centre-arrière** : Ligne 0, Colonnes T et $N - 1 - T$.

Le placement des Noirs est obtenu par symétrie centrale. Cette méthode garantit une ouverture équilibrée, évite le contact immédiat et favorise le contrôle des zones stratégiques.

1.3 Notation algébrique (F20)

Les cases sont repérées par une colonne (de a à j pour un plateau 10×10) et une ligne (de 1 à 10). Les coups suivent la notation algébrique standard.

Exemple : e2-e4/e6 signifie déplacer une amazone de e2 vers e4, puis tirer une flèche sur la case e6.

2 Structures de données

2.1 Module Bitboard

Conformément aux exigences F26 et F27 du cahier des charges, le plateau de jeu est représenté à l'aide de **bitboards**. Cette structure de données, issue des moteurs d'échecs [1], encode l'état du jeu sous forme d'entiers binaires, permettant des opérations en temps constant $O(1)$.

2.1.1 Définition et structure

Un *bitboard* est un entier de n^2 bits représentant un plateau de taille $n \times n$. Chaque bit correspond à une case unique du plateau :

- **bit = 1** → présence de l'objet sur cette case
- **bit = 0** → absence de l'objet sur cette case

L'état complet du jeu est représenté par **exactement trois bitboards distincts**, chacun dédié à un type d'objet unique :

Bitboard	Contenu (exclusif)
white_amazons	Positions des amazones blanches uniquement
black_amazons	Positions des amazones noires uniquement
blocked	Positions des cases bloquées (flèches) uniquement

TABLE 3 – Les trois bitboards représentant l'état du jeu

Règle fondamentale : Aucun bitboard ne mélange plusieurs types d'objets. Cette séparation stricte garantit l'intégrité des opérations binaires.

2.1.2 Indexation des cases

Chaque case du plateau possède un index unique calculé par la formule :

$$\text{index} = \text{ligne} \times n + \text{colonne}$$

où n est la taille du plateau, et $\text{ligne}, \text{colonne} \in [0, n - 1]$.

Case (affichée)	Coord. interne	Calcul ($n = 10$)	Index
a1	(0, 0)	$0 \times 10 + 0$	0
d4	(3, 3)	$3 \times 10 + 3$	33
j10	(9, 9)	$9 \times 10 + 9$	99

TABLE 4 – Correspondance notation algébrique ↔ index bitboard (plateau 10×10)

Note : L'affichage utilisateur (lignes 1 à 10, colonnes a-j) est une simple conversion de la représentation interne.

2.1.3 Combinaison des bitboards

Pour déterminer l'ensemble des cases occupées, on combine les trois bitboards par l'opération OR logique :

$$\text{occupied} = \text{white_amazons} \mid \text{black_amazons} \mid \text{blocked}$$

Cette opération produit un nouveau bitboard où chaque bit à 1 indique une case occupée, indépendamment du type d'objet.

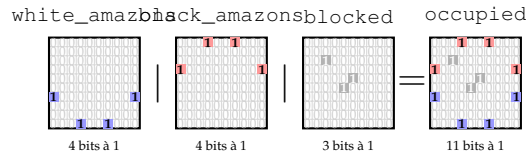


FIGURE 2 – Représentation binaire sur plateau 10×10 : 0 (case vide), 1 (case occupée). L’opération OR combine les bits.

2.1.4 Visualisation des opérations critiques

Les figures suivantes détaillent la mécanique interne des opérations de validation et de déplacement.

Validation d’un coup (Lecture) Pour vérifier si un coup est valide (ex : déplacement d1 vers d4), on utilise des masques binaires et l’opérateur AND. L’algorithme :

1. Agrège tous les obstacles (reines + flèches) via OR.
2. Applique un masque du chemin voulu.
3. Vérifie l’intersection via AND. Une valeur non nulle indique une collision.

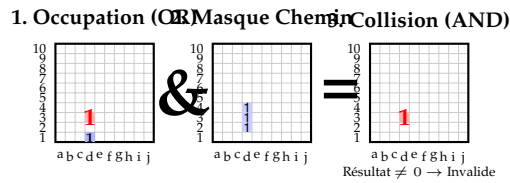


FIGURE 3 – Validation d’un coup : L’opération AND détecte la collision sur un plateau 10×10 .

Application d’un coup (Écriture) Le déplacement ne se fait pas par copie, mais par manipulation directe via XOR et OR. Pour déplacer une amazone de d1 (index 3) vers d4 (index 33) :

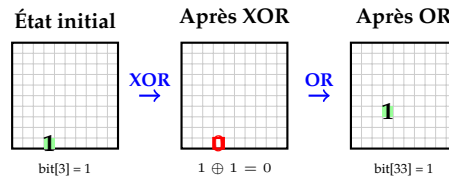


FIGURE 4 – Déplacement d’une amazone : XOR retire, OR place (plateau 10×10).

2.1.5 Algorithmes de manipulation (F27)

Note : Dans cette section, les exemples et formules utilisent un plateau de taille 10×10 (configuration standard). L’implémentation finale supportera toutes les tailles de 4×4 à 11×11 en paramétrant n dans les calculs d’index et les masques de bords.

Le cahier des charges impose cinq opérations. Voici leur conception détaillée.

Opération 1 : Vérifier si un coup est valide Algorithme :

1. Vérifier présence amazone en from : $(\text{my_amazons} \wedge (1 \ll \text{from})) \neq 0$
2. Vérifier alignement ligne/colonne/diagonale
3. Vérifier chemin libre : combiner obstacles occupé = white | black | blocked puis tester intersection
4. Vérifier flèche depuis to (arrow peut = from)

Opération 2 : Calculer les coups possibles Entrée : Couleur du joueur.

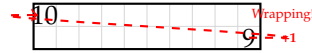
Sortie : Liste de tous les coups légaux (triplets from, to, arrow).

1. Arithmétique des index : Sur un bitboard 1D de largeur $L = 10$, les déplacements se traduisent par des décalages d’index constants. Comme l’index est $y \times 10 + x$, monter d’une ligne ajoute 10, se déplacer latéralement change l’unité :

— **Vertical** : Nord (+10), Sud (−10).

- **Horizontal** : Est (+1), Ouest (−1).
- **Diagonal** : Combinaison linéaire. Ex : NE = Nord + Est = +10 + 1 = +11. De même : NO (+9), SE (−9), SO (−11).

2. Problème du "Wrapping" (Effet Pac-Man) : L'indexation linéaire crée une fausse continuité entre les bords. Exemple : Index 9 (bord droit, ligne 0) +1 → Index 10 (bord gauche, ligne 1). Un déplacement simple vers l'Est provoquerait une téléportation de l'autre côté du plateau.



3. Solution par Masques de Bords : Le traitement des limites diffère selon l'axe :

- Axe Horizontal (Masques)** : Pour éviter le "Wrapping", nous utilisons des masques identifiant les colonnes critiques.
 - MASK_COL_A : Bits à 1 sur la colonne a (index 0, 10, ..., 90).
 - MASK_COL_J : Bits à 1 sur la colonne j (index 9, 19, ..., 99).
- Axe Vertical (Limites)** : Pas besoin de masque complexe. Si l'index calculé est < 0 ou ≥ 100 , on sort simplement du tableau (débordement haut/bas).

Algorithme de propagation : Pour chaque amazone à la position P et pour chaque direction D (offset) :

- Initialiser $cur = P$.
- Tant que la propagation est valide :
 - **Vérification Anti-Wrap (Horizontal)** :
 - Pour les directions ayant une composante Est (ex : +1, −9, +11) : si $cur \in \text{MASK_COL_J}$, alors **STOP**.
 - Pour les directions ayant une composante Ouest (ex : −1, +9, −11) : si $cur \in \text{MASK_COL_A}$, alors **STOP**.
 - Calculer $next = cur + D$.
 - **Vérification Verticale (Limites)** : Si $next < 0$ ou $next \geq 100$: **STOP**.
 - **Collision** : Si la case $next$ est occupée (test bit à 1) : **STOP**.
 - Sinon, ajouter $next$ aux coups valides et avancer ($cur = next$).
- Pour chaque destination to validée, relancer cet algorithme depuis to pour trouver les tirs de flèche $arrow$ (en considérant $from$ comme vide).

Opération 3 : Appliquer un coup Entrée : Un coup ($from, to, arrow$).

Effet : Modifie l'état du plateau (voir Figure 4).

Algorithme (Ordre strict requis) :

- 1. Libérer la case de départ (XOR)** : Cette étape doit être réalisée **avant** de tirer la flèche, car la flèche peut légalement atterrir sur la case $from$ qui vient d'être libérée.

$$\text{amazons} = \text{amazons} \oplus (1 \ll \text{from})$$

- 2. Occuper la case d'arrivée (OR)** :

$$\text{amazons} = \text{amazons} \mid (1 \ll \text{to})$$

- 3. Tirer la flèche (OR)** : La case $arrow$ est marquée comme bloquée. Si $arrow == from$, cela fonctionne correctement car l'étape 1 a déjà libéré la place.

$$\text{blocked} = \text{blocked} \mid (1 \ll \text{arrow})$$

Opération 4 : Calculer les scores Score = $M_{\text{joueur}} - M_{\text{adversaire}}$ où M = nombre de coups possibles.

Opération 5 : Fin de partie Le joueur a perdu si aucun coup n'est possible (Opération 2 retourne liste vide).

2.1.6 Résumé des opérations bit à bit

Opération	Notation	Utilisation
Shift gauche	$1 \ll n$	Créer un masque pour la position n
OR	$a b$	Placer une pièce ou bloquer une case
XOR	$a \oplus b$	Retirer une pièce (inverser un bit)
AND	$a \wedge b$	Tester si une case est occupée

TABLE 5 – Récapitulatif des opérations bit à bit

2.2 Architecture logicielle

L'architecture du projet suit le pattern MVC (Modèle-Vue-Contrôleur). La figure 1 présente le diagramme de classes.

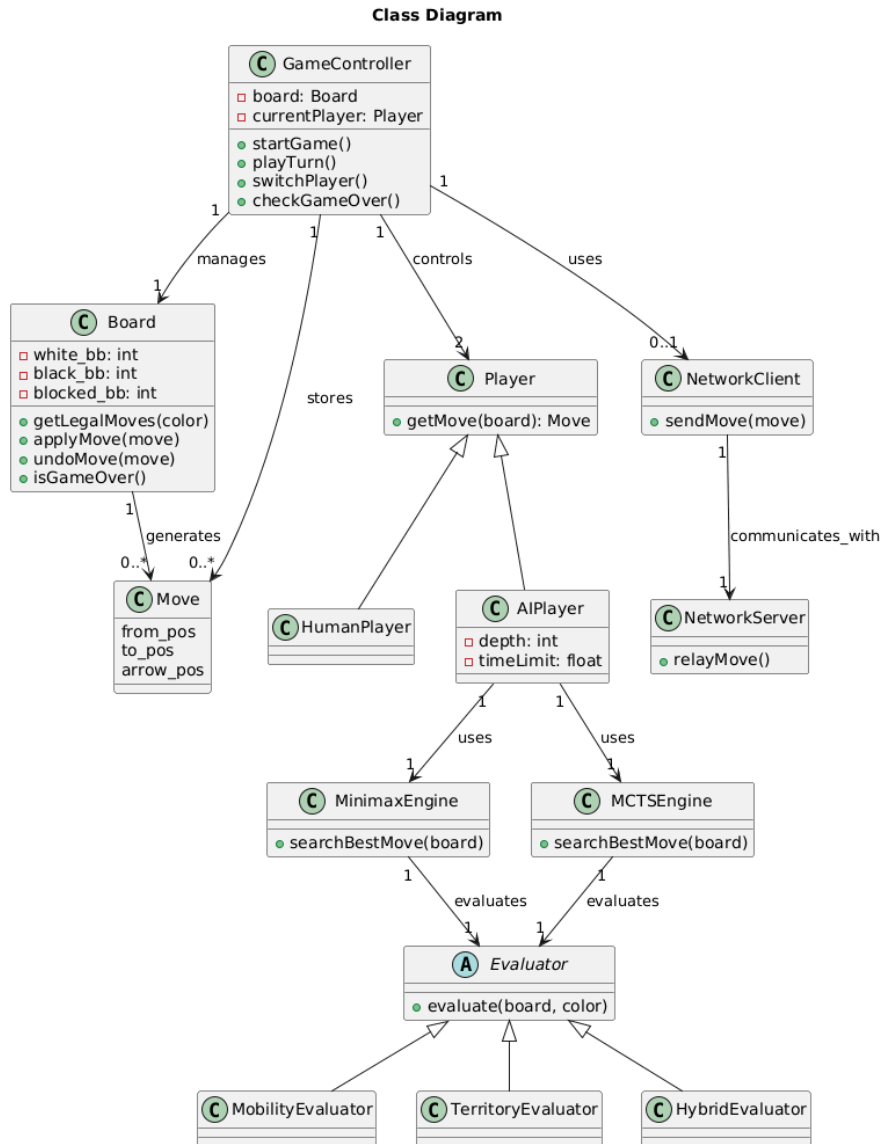


FIGURE 5 – Diagramme de classes UML [5]

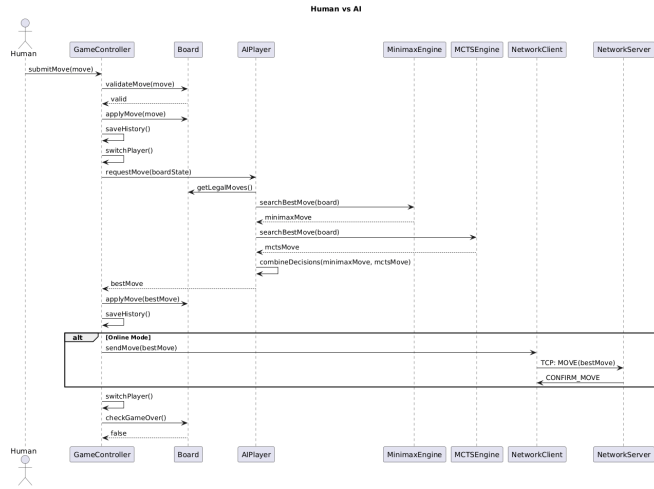


FIGURE 6 – Humain vs AI Player

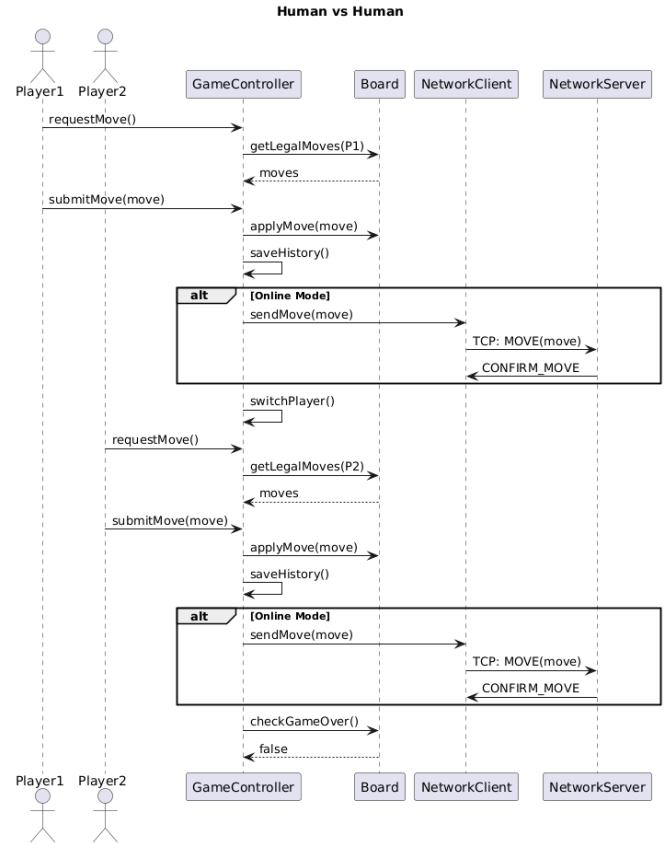


FIGURE 7 – Humain vs Humain

3 Intelligence artificielle (IA hybride)

Le joueur artificiel repose sur une chaîne de décision commune :

- **Génération des coups légaux** : `Board.getLegalMoves(color)` produit une liste de coups `Move(from, to, arrow)`.
- **Recherche** : l'IA utilise deux moteurs complémentaires : **Minimax avec élagage $\alpha\beta$** et **Monte Carlo Tree Search (MCTS)**.
- **Évaluation** : chaque position est notée via `Evaluator.evaluate(board, color)` (mobilité, territoire ou hybride).
- **Fusion** : l'`AIPlayer` combine les propositions de Minimax et MCTS pour produire un `bestMove`.

Pipeline global :

$$Board \rightarrow getLegalMoves \rightarrow (Minimax + MCTS) \rightarrow Evaluator \rightarrow bestMove$$

3.1 Minimax avec élagage $\alpha\beta$

Minimax modélise la partie comme un arbre alternant : **MAX** (notre tour, maximiser le score) et **MIN** (tour adverse, minimiser le score).

La recherche est limitée par une profondeur ou un budget temps. Si `depth = 0` ou fin de partie, on renvoie :

$$score = evaluate(board, color)$$

Pour explorer l'arbre, l'algorithme applique un coup avec `applyMove(move)` puis revient en arrière avec `undoMove(move)`. L'élagage $\alpha\beta$ évite d'explorer des branches inutiles lorsque :

$$\alpha \geq \beta$$

3.2 Fonctions d'évaluation

Trois heuristiques sont implémentées via héritage de `Evaluator` :

Mobilité :

$$score_{mob}(s) = |Moves(s, joueur)| - |Moves(s, adversaire)|$$

Territoire :

$$score_{terr}(s) = \#cases_{nous} - \#cases_{lui}$$

Hybride :

$$score_{hyb}(s) = \lambda \cdot score_{mob}(s) + (1 - \lambda) \cdot score_{terr}(s)$$

avec $\lambda \in [0, 1]$.

3.3 Monte Carlo Tree Search (MCTS)

MCTS [2] explore l'arbre par simulations successives :

- Sélection (UCT)
- Expansion
- Simulation (rollout)
- Rétropropagation

Le critère UCT [4] équilibre exploration et exploitation :

$$UCT_i = \frac{w_i}{n_i} + C \sqrt{\frac{\ln N}{n_i}}$$

Le coup final correspond au nœud racine le plus robuste (meilleure moyenne ou plus visité).

4 Jeu en réseau

Point critique identifié : La gestion du réseau constitue une partie technique sensible du projet, susceptible de bloquer son avancement si elle n'est pas anticipée correctement. Pour cette raison, une étude des différentes bibliothèques réseau disponibles en Python a été réalisée dès les premières phases de conception.

4.1 Bibliothèques Python pour le réseau

Bibliothèque	Usage	Avantage
<code>socket</code>	Connexions TCP/UDP	Standard, bas niveau
<code>asyncio</code>	Entrées/Sorties asynchrones	Gestion multi-clients sans threads

TABLE 6 – Bibliothèques réseau Python étudiées

Choix retenu : L'architecture réseau repose sur l'utilisation conjointe des bibliothèques `socket` [6] et `asyncio` afin d'assurer une gestion asynchrone et efficace des connexions.

Justification :

- Gestion efficace de plusieurs clients simultanément
- Absence de surcharge liée au multi-threading (verrou global d'interpréteur Python – GIL)
- Compatibilité avec la structure des données du jeu (bitboards facilement sérialisables)

4.2 Architecture client–serveur

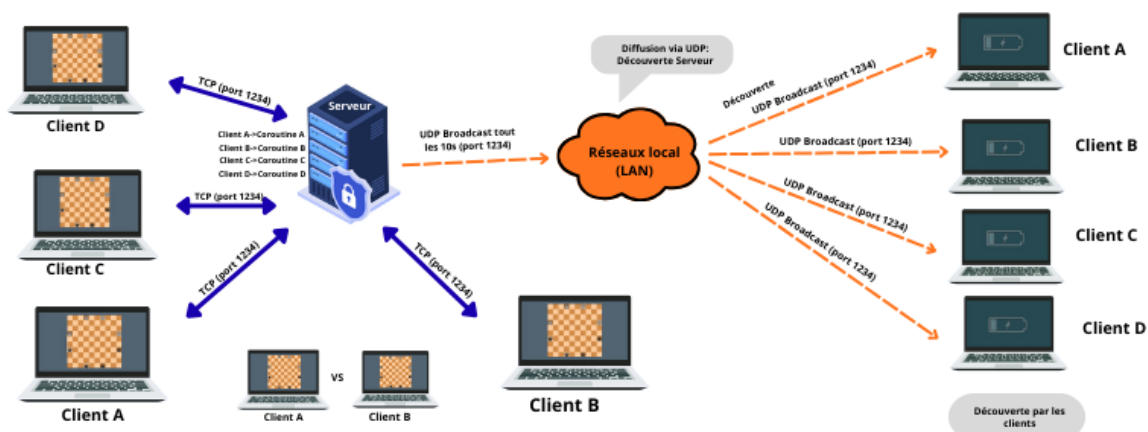


FIGURE 8 – Architecture réseau client–serveur du jeu

Cette architecture repose sur une séparation claire entre la phase de découverte du serveur via UDP et la phase de jeu reposant sur des connexions TCP persistantes.

- Découverte automatique du serveur sur le réseau local via des messages UDP en broadcast (port 12346)
- Connexion des clients au serveur par TCP (port 12345 par défaut)
- Gestion simultanée de plusieurs clients et parties grâce à une boucle événementielle `asyncio`
- Système d’invitations entre joueurs avec gestion des délais d’expiration

5 Besoins fonctionnels

Cette section liste l’ensemble des exigences fonctionnelles du cahier des charges.

ID	Catégorie	Description synthétique
F1–F3	Lancement	Options CLI, config utilisateur, internationalisation
F4–F9	Modes de jeu	CLI, Blitz, Contest, GUI, IA, taille variable
F10–F14	Gestion du jeu	Validation, gestion erreurs, Undo/Redo
F15–F20	Interface CLI	Shell interactif, historique, notation algébrique
F21–F25	Sauvegarde	Format ASCII, paramètres, état et historique
F26–F27	Structures	Bitboards + algorithmes cœur du moteur
F28–F30	Interface GUI	Menus, raccourcis, Drag’n’Drop
F31–F37	Intelligence artificielle	Minimax, MCTS, évaluation, ML
F38–F40	Réseau	Découverte UDP, TCP multi-clients, invitations

TABLE 7 – Synthèse des besoins fonctionnels (F1–F40)

6 Besoins non fonctionnels

Exigence	Description
Performance	L’IA doit répondre en moins de 5 secondes par défaut
Portabilité	Fonctionne sur GNU/Linux (cible principale)
Maintenabilité	Code modulaire, couverture tests $\geq 85\%$
Robustesse	Aucun crash, messages d’erreur explicites
Internationalisation	Support anglais (défaut) et français

TABLE 8 – Exigences non fonctionnelles

Conclusion

Ce rapport préliminaire présente une analyse approfondie des choix techniques et méthodologiques pour le développement du jeu des Amazones. Plusieurs aspects clés ont été identifiés et traités :

Architecture et structures de données : L'utilisation des bitboards comme structure centrale offre des performances optimales en temps constant $O(1)$ pour toutes les opérations critiques (validation, génération de coups, application). Cette approche, inspirée des moteurs d'échecs professionnels, permet une réduction mémoire de 90% tout en garantissant l'efficacité du moteur.

Intelligence artificielle : L'implémentation hybride combinant Minimax avec élagage $\alpha\beta$ et Monte Carlo Tree Search (MCTS) permet d'exploiter les forces complémentaires de ces deux approches. Le système d'évaluation heuristique (mobilité, territoire) fournit une base solide pour l'analyse stratégique.

Architecture logicielle : Le pattern MVC assure une séparation claire des responsabilités, facilitant la maintenance et l'évolution du code. L'architecture réseau basée sur `socket` et `asyncio` permet une gestion asynchrone efficace pour le mode multi-joueurs.

Qualité et robustesse : Les exigences de couverture de tests ($\geq 85\%$), la documentation exhaustive via Sphinx, et la gestion rigoureuse des erreurs garantissent un logiciel fiable et maintenable.

Les prochaines étapes consistent à implémenter le module bitboard complet et la logique de jeu de base lors du Sprint 0, avant de passer au développement des fonctionnalités avancées (IA, réseau, interface graphique GTK4).

Références

- [1] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.
- [2] Cameron Browne, Edward Powley, Daniel Whitehouse, Simon Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1) :1–43, 2012.
- [3] Robert A. Hearn. Amazons is pspace-complete. *Computing Research Repository (CoRR)*, abs/cs/0502013, 2005.
- [4] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *Machine Learning : ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293, Berlin, Heidelberg, 2006. Springer.
- [5] PlantUML Team. Plantuml : Open-source tool that uses simple textual descriptions to draw diagrams, 2024. Accessed : 2026-01-29.
- [6] Python Software Foundation. `socket` — *Low-level networking interface*. Python Software Foundation, 2024. Accessed : 2026-01-29.
- [7] Jiaying Song and Martin Müller. An enhanced solver for the game of amazons. *IEEE Transactions on Computational Intelligence and AI in Games*, 7(1) :16–27, 2015.